

Informe de Pruebas Unitarias y Aseguramiento de la Calidad

Integrantes:

- Sebastián Carrera

Asignatura: Fullstack III

Introducción.....	2
Objetivos de las pruebas.....	2
Herramientas utilizadas.....	3
Backend.....	3
Frontend.....	3
Estrategia de pruebas.....	4
Pruebas unitarias.....	4
Pruebas de controladores.....	4
Pruebas de persistencia.....	4
Pruebas de integración.....	5
Pruebas del frontend.....	5
Resultados consolidados.....	6
Resultados por componente.....	6
Frontend.....	6
Microservicio de usuarios.....	7
Microservicio de autenticación.....	7
Microservicio de campañas.....	8
Microservicio de donaciones.....	8
Backend for Frontend.....	9
API Gateway.....	10
Automatización de las pruebas.....	10
Ubicación de las evidencias.....	11
Frontend.....	11
Microservicio de usuarios.....	11
Microservicio de autenticación.....	11
Microservicio de campañas.....	11
Microservicio de donaciones.....	12
Backend for Frontend.....	12
API Gateway.....	12
Interpretación de los resultados.....	12
Incidencias detectadas durante las pruebas.....	13
Mejoras futuras.....	13
Conclusión.....	14

Introducción

El presente informe documenta la estrategia de pruebas unitarias y de integración implementada en la plataforma Donaton. El objetivo de estas pruebas es verificar el comportamiento de los componentes frontend y backend, comprobar la correcta comunicación entre las capas de la aplicación y disponer de métricas objetivas sobre la calidad del código desarrollado.

La solución está compuesta por un frontend desarrollado con Next.js y TypeScript, un Backend for Frontend, un API Gateway y cuatro microservicios desarrollados con Java y Spring Boot:

- Microservicio de usuarios.
- Microservicio de autenticación.
- Microservicio de campañas.
- Microservicio de donaciones.
- Backend for Frontend.
- API Gateway.
- Frontend web.

Las pruebas se ejecutan de manera automatizada mediante un script centralizado, permitiendo validar todos los componentes del proyecto antes de generar una versión entregable.

Objetivos de las pruebas

Los principales objetivos del proceso de pruebas fueron:

1. Verificar que las reglas de negocio se ejecuten correctamente.
2. Comprobar las operaciones de creación, consulta, actualización y eliminación de datos.
3. Validar el comportamiento de los controladores REST.
4. Comprobar la persistencia mediante repositorios JPA.
5. Verificar la generación y validación de tokens JWT.
6. Validar la comunicación realizada por el Backend for Frontend.
7. Comprobar el enrutamiento y la configuración CORS del API Gateway.
8. Validar las funciones de sesión, consumo de API y servicios del frontend.
9. Detectar regresiones antes de desplegar la solución mediante Docker y Kubernetes.
10. Obtener métricas de cobertura de código mediante JaCoCo y Vitest Coverage.

Herramientas utilizadas

Backend

Para los componentes backend se utilizaron las siguientes herramientas:

- JUnit: framework utilizado para definir y ejecutar las pruebas.
- Mockito: herramienta utilizada para simular dependencias y aislar las clases bajo prueba.
- Spring Boot Test: permite cargar el contexto de Spring y realizar pruebas de integración.
- MockMvc: permite probar controladores HTTP sin iniciar manualmente un servidor externo.
- H2 Database: base de datos en memoria utilizada para probar los repositorios JPA sin afectar las bases de datos reales de Neon.
- JaCoCo: herramienta utilizada para medir la cobertura de instrucciones, líneas, métodos y ramas.
- Gradle: herramienta encargada de compilar el proyecto y ejecutar las pruebas.

Frontend

Para el frontend se utilizaron:

- Vitest: framework principal de pruebas.
- React Testing Library: permite probar componentes desde la perspectiva del usuario.
- JSDOM: entorno simulado de navegador.
- Vitest Coverage V8: herramienta de medición de cobertura.
- TypeScript Compiler: validación estática de tipos mediante tsc.
- ESLint: análisis estático y detección de problemas de calidad.
- Next.js Build: validación de la compilación optimizada para producción.

Estrategia de pruebas

La estrategia implementada combina diferentes niveles de prueba.

Pruebas unitarias

Las pruebas unitarias validan clases o funciones de forma aislada. Las dependencias externas son reemplazadas mediante mocks.

Se aplicaron principalmente en:

- Servicios de negocio de los microservicios.
- Servicio de autenticación.
- Generación y validación de JWT.
- BackendGatewayService del BFF.
- Funciones de sesión del frontend.
- Cliente HTTP del frontend.
- Servicios frontend de usuarios, autenticación, campañas y donaciones.

Pruebas de controladores

Estas pruebas verifican:

- Rutas HTTP.
- Códigos de estado.
- Validación de datos.
- Serialización y deserialización JSON.
- Invocación de los servicios correspondientes.
- Encabezados HTTP.

Se utilizaron pruebas con MockMvc en los controladores de usuarios, autenticación, campañas, donaciones y BFF.

Pruebas de persistencia

Las pruebas de repositorios verifican la interacción con la base de datos mediante Spring Data JPA.

Para evitar conexiones con las bases reales, se utilizó H2 en memoria. Esto permite ejecutar pruebas repetibles, independientes y seguras.

Se validaron operaciones como:

- Guardar entidades.
- Buscar por identificador.
- Buscar usuarios mediante correo electrónico.
- Comprobar la existencia de correos registrados.
- Consultar todos los registros.
- Actualizar datos.
- Eliminar registros.

Pruebas de integración

Se realizaron pruebas donde se carga el contexto real de Spring Boot y se comprueba la integración entre diferentes componentes.

En el API Gateway se implementó una prueba de enrutamiento utilizando un servidor HTTP simulado. Esta prueba permitió comprobar que:

- Las solicitudes dirigidas a /api/** son enviadas al BFF.
- El Gateway propaga correctamente las respuestas exitosas.
- Los códigos de error entregados por el servicio downstream se conservan.
- La configuración CORS permite solicitudes desde http://localhost:3000.

Pruebas del frontend

En el frontend se probaron:

- Renderización de formularios.
- Validación del inicio de sesión.
- Validación del registro.
- Control de contraseñas no coincidentes.
- Persistencia y eliminación de sesión.
- Recuperación del token JWT.
- Construcción de solicitudes HTTP.
- Incorporación del encabezado Authorization.
- Lectura de respuestas JSON y texto.
- Extracción de mensajes de error.
- Consumo de servicios de autenticación, usuarios, campañas y donaciones.

Resultados consolidados

La ejecución automatizada completa obtuvo los siguientes resultados:

Componente	Pruebas Ejecutadas	Fallos	Resultado	Cobertura Principal
Frontend	42	0	Aprobado	25% Global
MS-User	16	0	Aprobado	84%
MS-Auth	15	0	Aprobado	100%
MS-Campaign	19	0	Aprobado	100%
MS-Donation	19	0	Aprobado	100%
BFF	39	0	Aprobado	100%
Api Gateway	13	0	Aprobado	100%
Total	163	0	Aprobado	

La ejecución consolidada finalizó satisfactoriamente, sin pruebas fallidas ni errores.

Resultados por componente

Frontend

El frontend cuenta con 42 pruebas distribuidas entre páginas, bibliotecas y servicios. Las bibliotecas principales alcanzaron una cobertura completa.

La cobertura global del frontend alcanzó:

- Instrucciones o sentencias: 25,00 %.
- Ramas: 22,80 %.
- Funciones: 22,79 %.
- Líneas: 25,00 %.

La cobertura global es inferior a la cobertura de los módulos críticos porque el reporte también incluye numerosos componentes visuales que todavía no poseen pruebas específicas.

No obstante, la lógica relacionada con autenticación, almacenamiento de sesión, construcción de solicitudes HTTP y comunicación con el backend cuenta con cobertura completa.

Microservicio de usuarios

El microservicio de usuarios ejecutó 16 pruebas:

- 1 prueba de carga de contexto.
- 2 pruebas de repositorio.
- 13 pruebas de servicio.

La cobertura total alcanzó un 84 %. La capa de servicio obtuvo un 100 % de cobertura, mientras que el porcentaje global disminuye porque el controlador no fue incluido completamente dentro del conjunto de clases cubiertas por las métricas finales.

Las pruebas validaron:

- Creación de usuarios.
- Codificación de contraseñas.
- Rechazo de correos duplicados.
- Consulta de usuarios.
- Búsqueda por correo electrónico.
- Actualización de perfil.
- Eliminación de usuarios.
- Manejo de usuarios inexistentes.
- Obtención de datos internos para autenticación.
- Persistencia mediante JPA.

Microservicio de autenticación

El microservicio de autenticación ejecutó 15 pruebas:

- 1 prueba de contexto.
- 3 pruebas del controlador.
- 4 pruebas del servicio de autenticación.
- 7 pruebas del servicio JWT.

La cobertura alcanzó un 100 % tanto en instrucciones como en ramas.

Las pruebas validaron:

- Credenciales correctas.
- Rechazo de contraseñas incorrectas.
- Generación de tokens JWT.
- Obtención del correo desde el token.
- Obtención del rol.
- Validación de tokens correctos.
- Rechazo de tokens alterados.
- Respuesta de validación.
- Endpoints de login y validación.

Microservicio de campañas

El microservicio de campañas ejecutó 19 pruebas:

- 1 prueba de contexto.
- 4 pruebas del repositorio.
- 9 pruebas del servicio.
- 5 pruebas del controlador.

La cobertura alcanzó un 100 % en las capas de servicio y controlador consideradas en el reporte.

Las pruebas validaron:

- Creación de campañas.
- Estado inicial ACTIVA.
- Monto recaudado inicial.
- Listado de campañas.
- Consulta por identificador.
- Actualización.
- Eliminación.
- Manejo de campañas inexistentes.
- Persistencia JPA.
- Respuestas HTTP del controlador.

Microservicio de donaciones

El microservicio de donaciones ejecutó 19 pruebas:

- 1 prueba de contexto.
- 4 pruebas del repositorio.

- 9 pruebas del servicio.
- 5 pruebas del controlador.

La cobertura alcanzó un 100 % en las capas incluidas.

Las pruebas verificaron:

- Creación de donaciones.
- Asignación del estado inicial PENDIENTE.
- Conversión de tipos de donante y recurso.
- Listado de donaciones.
- Consulta por identificador.
- Actualización.
- Eliminación.
- Manejo de donaciones inexistentes.
- Persistencia JPA.
- Respuestas HTTP del controlador.

Backend for Frontend

El BFF ejecutó 39 pruebas:

- 1 prueba de contexto.
- 15 pruebas del controlador.
- 20 pruebas de BackendGatewayService.
- 3 pruebas del manejador de excepciones.

La cobertura alcanzó un 100 % en instrucciones y ramas.

Las pruebas validaron:

- Registro de usuarios.
- Inicio de sesión.
- Obtención de datos de autenticación.
- Comunicación con MS User y MS Auth.
- Consulta del usuario autenticado.
- Validación del token.
- Creación de donaciones asociadas al usuario autenticado.
- Consulta de donaciones.
- Creación, consulta, actualización y eliminación de campañas.
- Propagación de solicitudes hacia los microservicios.

- Manejo de cuerpos de respuesta inválidos.
- Manejo de identificadores faltantes.
- Manejo de errores de servicios downstream.
- Validación de solicitudes recibidas por los controladores.

API Gateway

El API Gateway ejecutó 13 pruebas:

- 6 pruebas de aplicación y configuración.
- 4 pruebas de CORS.
- 3 pruebas de enrutamiento.

La configuración propia del Gateway alcanzó un 100 % de cobertura.

Las pruebas validaron:

- Carga del contexto de Spring.
- Existencia de la ruta hacia el BFF.
- Enrutamiento de /api/**.
- Propagación de respuestas HTTP.
- Propagación de errores 404.
- Configuración del origen permitido.
- Métodos permitidos.
- Encabezados permitidos.
- Solicitudes preflight de CORS desde el frontend.

Automatización de las pruebas

Para simplificar el proceso de validación se implementó el script:

```
scripts/run-all-tests.sh
```

Este script ejecuta de manera centralizada:

1. Comprobación de tipos TypeScript.
2. ESLint.
3. Pruebas del frontend.
4. Cobertura del frontend.

5. Compilación de producción de Next.js.
6. Pruebas de MS User.
7. Pruebas de MS Auth.
8. Pruebas de MS Campaign.
9. Pruebas de MS Donation.
10. Pruebas del BFF.
11. Pruebas del API Gateway.
12. Generación de los reportes JaCoCo.

La ejecución se realiza desde la raíz del proyecto:

```
./scripts/run-all-tests.sh
```

Adicionalmente, se implementó el script:

```
scripts/generate-test-summary.py
```

Este procesa automáticamente los resultados XML de Gradle y los archivos JSON de cobertura del frontend, generando:

```
docs/testing/test-summary.md  
docs/testing/test-summary.json
```

Esto reduce errores de transcripción y permite mantener actualizadas las métricas del proyecto.

Ubicación de las evidencias

Frontend

Reporte de cobertura:

```
frontend/coverage/index.html
```

Microservicio de usuarios

```
backend/ms-user/build/reports/tests/test/index.html  
backend/ms-user/build/reports/jacoco/test/html/index.html
```

Microservicio de autenticación

```
backend/ms-auth/build/reports/tests/test/index.html  
backend/ms-auth/build/reports/jacoco/test/html/index.html
```

Microservicio de campañas

```
backend/ms-campaign/build/reports/tests/test/index.html  
backend/ms-campaign/build/reports/jacoco/test/html/index.html
```

Microservicio de donaciones

backend/ms-donation/build/reports/tests/test/index.html

backend/ms-donation/build/reports/jacoco/test/html/index.html

Backend for Frontend

backend/bff/build/reports/tests/test/index.html

backend/bff/build/reports/jacoco/test/html/index.html

API Gateway

backend/api-gateway/build/reports/tests/test/index.html

backend/api-gateway/build/reports/jacoco/test/html/index.html

Interpretación de los resultados

Los resultados obtenidos muestran que la lógica crítica del backend se encuentra ampliamente cubierta. Los servicios de autenticación, campañas, donaciones, BFF y API Gateway alcanzaron un 100 % de cobertura dentro de las clases seleccionadas para el análisis.

El microservicio de usuarios alcanzó un 84 % global y un 100 % en su capa de servicio, lo que indica que las reglas de negocio principales fueron probadas satisfactoriamente.

El frontend presenta una cobertura global del 25 %. Este porcentaje se explica porque el reporte considera todos los componentes visuales, aunque muchos de ellos no contienen reglas de negocio complejas y todavía no poseen pruebas individuales.

Las funciones críticas del frontend relacionadas con sesión, autenticación, consumo HTTP y comunicación con el BFF alcanzaron un 100 % de cobertura.

Por lo tanto, la cobertura obtenida permite afirmar que las áreas de mayor riesgo funcional cuentan con pruebas automatizadas, aunque existe margen de mejora en los componentes visuales y hooks del frontend.

Incidencias detectadas durante las pruebas

La implementación de las pruebas permitió detectar y corregir distintos problemas:

- Dependencia accidental de credenciales de producción durante pruebas.
- Necesidad de utilizar una base H2 en memoria.
- Aserciones desactualizadas en la actualización de usuarios.
- Problemas de compatibilidad entre versiones de Vitest y su proveedor de cobertura.
- Configuración duplicada de dependencias.
- Falta de pruebas iniciales en algunos microservicios.
- Problemas de inicialización de Mockito en el BFF.
- Configuración incompleta de las propiedades dinámicas del API Gateway.
- Ausencia del identificador de ruta en las pruebas del Gateway.
- Advertencias de ESLint por actualizaciones de estado dentro de efectos.
- Inclusión accidental de directorios de cobertura en el análisis de ESLint.

La corrección de estas incidencias aumentó la estabilidad, reproducibilidad y mantenibilidad del proyecto.

Mejoras futuras

Como trabajo futuro se recomienda:

1. Incorporar pruebas para los componentes visuales del frontend.
2. Probar los hooks useCampaigns, useDonations, useDashboard y useProfile.
3. Incorporar pruebas de accesibilidad.
4. Agregar pruebas end-to-end con Playwright o Cypress.
5. Validar la aplicación completa desplegada en Kubernetes.
6. Incorporar pruebas de rendimiento.
7. Establecer umbrales mínimos de cobertura.
8. Ejecutar automáticamente las pruebas en cada integración de código.
9. Implementar pruebas de seguridad sobre autenticación y autorización.
10. Validar escenarios de indisponibilidad de microservicios y mecanismos de resiliencia.

Conclusión

La estrategia aplicada permitió ejecutar un total de 163 pruebas automatizadas sin fallos.

Los resultados demuestran que la solución cuenta con pruebas sobre las reglas de negocio, persistencia, controladores REST, autenticación JWT, comunicación entre microservicios, enrutamiento del API Gateway y funciones críticas del frontend.

La automatización mediante Gradle, Vitest, JaCoCo y scripts propios permite repetir el proceso de validación de manera consistente antes de cada entrega o demostración.

Aunque la cobertura de los componentes visuales del frontend puede seguir mejorándose, las áreas críticas relacionadas con datos, autenticación, comunicación y lógica de negocio presentan una cobertura sólida.

En consecuencia, el proceso de pruebas contribuye directamente a la estabilidad, mantenibilidad y confiabilidad de la plataforma Donaton.